

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO4 – Articulation (BL3)	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	30% of CIA	Total Marks:	100
CO4 Statement:	Apply hashing strategies for efficient data storage and sorting algorithms for arrangement of data.		
Student Name:	Sasikumar Megha	Reg. No.:	192471038
Section / Batch:	3 rd C&S	Date:	26/08/2026

Code of ConductI Sasikumar Megha (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S. Megha**Instructions**

- This test contains 5 Comparative Analysis questions. Total: 100 Marks.
- When a student answer using comparative analysis should be based on four requirements: time complexity, space complexity advantages & limitations, real-world scenarios and operations.
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Hashing	Linear Probing & Separate Chaining	1	20
Sorting	Merge Sort & Quick Sort	2	20
Hashing	Quadratic Probing and Double Hashing	3	20
Sorting Algorithms	Complexity Analysis	4	20
Quick Sort	Recursive and Iterative implementations	5	20
GRAND TOTAL:			100 Marks

Annexure A – Comparative Analysis

1. Compare Linear Probing and Separate Chaining for collision handling in hashing based on: (20 Marks)
 - a) Clustering effect
 - b) Memory usage
 - c) Performance under high load factor
 - d) Which method is more suitable for large-scale applications and why?
2. Compare quick sort and merge sort for sorting a large linked list of 10 million nodes. Analyze memory requirements (in-place vs. extra space), cache behavior, and time complexity. Which algorithm is preferable for linked lists and why? How does the answer change for arrays? (20 Marks)
3. Compare Quadratic Probing and Double Hashing in terms of: (20 Marks)
 - a) Clustering issues
 - b) Collision resolution efficiency
 - c) Suitability for dense hash tables
4. Compare Best-case and Worst-case time complexity of sorting algorithms based on the following: (20 Marks)
 - a) Practical significance
 - b) Impact on algorithm selection
5. Compare Recursive and Iterative implementations of Quick Sort in terms of: (20 Marks)
 - a) Space complexity (stack usage)
 - b) Ease of implementation
 - c) Risk of stack overflow

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Understanding	20	Demonstrates complete and accurate understanding of all data structures with advanced insights	Shows clear understanding with minor gaps	Basic understanding; some misconceptions present	Limited or incorrect understanding
Comparative Analysis Depth	20	Thorough, multi-dimensional comparison with strong justification and critical evaluation	Good comparison with relevant factors considered	Limited comparison; lacks depth or justification	Minimal or incorrect comparison
Use of Parameters (Time/Space/Operations)	30	All parameters analysed accurately with complexity discussion	Most parameters covered correctly	Few parameters considered; some inaccuracies	Parameters missing or incorrect
Critical Thinking	20	Strong justification of why one structure is better in given contexts	Some justification present	Limited reasoning	No critical reasoning
Clarity	10	Exceptionally well-structured, logical flow, clear tables/diagrams	Well-organized with minor issues	Some organization issues; lacks clarity	Poor structure and readability

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: 26/08/26	Date: _____	Date: _____

Co 4

Sasikumar
Megha
192471038

- i) Compare Linear Probing and separate chaining for collision handling in hashing based on:
- clustering effect
 - Memory usage
 - Performance under high load factor
 - which method is more suitable for large-scale application and why?

Basic	Linear Probing	Separate chaining
a) clustering effect	Suffers from primary clustering consecutive occupied slots form long clusters.	No primary clustering because each hash table index has its own chain.
b) Memory usage	uses only the hash table array, so memory overhead is low.	Requires extra memory for linked-list nodes / pointers so, memory usage is higher
c) High load factor	Performance decreases significantly as load factor approaches 1. More probes are required.	Handles high load factors better because multiple elements can exist in the same bucket.

d) Large-scale applications.	suitable when the table has a relatively low load factor and memory/cache efficiency is important	Generally more suitable for large-scale applications because it handles collisions and high load factors more effectively.
------------------------------	---	--

Conclusion:

Separate chaining is generally preferred for large-scale applications because it provides better performance when many elements are stored and avoids primary clustering. Linear Probing is simpler and more memory-efficient but works best when the load factor is kept relatively low.

2) Compare quick sort and merge sort for sorting a large scale linked list of 10 million nodes. Analyze memory requirements, cache behavior, and time complexity. Which algorithm is preferable for linked list and why? How does the answer change for arrays?

	Basic	Quick Sort	Merge Sort
Time complexity		Average: $O(n \log n)$ worst: $O(n^2)$	Best/Average/worst: $O(n \log n)$

Memory	can be impleted in-place but recursion requires stack space.	Required $O(\log n)$ stack space for linked-list recursive implementation. merging can be done without an extra array.
Cache behavior	Generally poor for linked lists because nodes are not stored contiguously.	Better suited to linked lists because it accesses nodes sequentially during merging.
Linked-list operations	partitioning is more difficult because random access is unavailable.	splitting and merging can be efficiently performed using pointers.
stability	usually not stable	stable
Large linked list	less suitable	highly suitable.

for 10 million linked-list nodes Merge sort is preferable.

- 1) It guarantees $O(n \log n)$ time
- 2) It does not require random access
- 3) splitting can be performed using slow/fast pointers.
- 4) Merging can be done by changing pointers without copying the entire list.
- 5) It avoids Quick sort's $O(n^2)$ worst case.

6) It is suitable for very large linked lists.

3. Quadratic Probing vs Double hashing.

Basic

Quadratic Probing

Double hashing.

a) clustering issues

eliminates primary clustering but can suffer from secondary clustering.

Minimizes both Primary and secondary clustering.

b) collision resolution efficiency

Better than linear probing but elements with the same initial hash value follow the same probing sequence.

Generally more efficient because a second hash function generates a different probe sequence.

c) Dense hash tables

Performance decreases as load factor becomes high

usually perform better than quadratic probing in dense tables, although very high load factor still reduce performance.

d) Probe sequence

$$h(K) + i^2$$

$$h_1(K) + i \times h_2(K)$$

complexity

Average approximately $O(1)$; worst $O(n)$

Average approximately $O(1)$; worst $O(n)$

Best-case Vs worst case Time complexity of sorting Algorithms.

Aspect	Best case	worst case
Meaning	Minimum time required for favorable input	Maximum time required for unfavorable input
Practical Significance	show how fast an algorithm can perform under ideal conditions	shows the performance guarantee when input is difficult.
Algorithm selection	useful when input are useful partially sorted or favorable	Important when predictable performance is required.
Risk	May not represent normal performance	Helps identify algorithms that can become extremely slow

ex:

Algorithm	Best case	worst case
Bubble sort	$O(n)$	$O(n^2)$
selection sort	$O(n^2)$	$O(n^2)$
Insertion sort	$O(n)$	$O(n^2)$
Merge sort	$O(n \log n)$	$O(n \log n)$
Quick sort	$O(n \log n)$	$O(n^2)$
Heap sort	$O(n \log n)$	$O(n \log n)$

for optimized Bubble sort

a) Practical significance

Best-case complexity shows the potential performance when the input is favorable. for example, Insertion sort is $O(n)$ when the data is already sorted.

worst-case complexity is important when the application must guarantee acceptable performance. for example, Quick sort can degrade to $O(n^2)$ with poor pivot selection.

b) Impact on algorithm selection

The choice depends on the application.

- * If data is nearly sorted $\rightarrow$ Insertion sort.
- * If guaranteed $O(n \log n)$ is required $\rightarrow$ Merge sort
- * If average practical performed is imp $\rightarrow$ Quick sort
- * If stable sorting is required $\rightarrow$ Merge sort.

Recursive Vs Iterative Quick sort

Basic

Recursive Quick sort

Iterative Quick sort

a) Space complexity

uses recursion stack. Average $O(\log n)$
worst $O(n)$

uses an explicit stack. Average $O(\log n)$; worst can be $O(n)$

b) Ease of implementation

simple and natural to implement

More complex because stack management is manual

c) stack overflow risk

Higher, especially with poor pivot selection

lower if the explicit stack is carefully managed.

Readability

easy to understand

slightly harder to understand

control over memory

less control over function-call stack

more control because programmer manages the stack.

Recursive Quick sort

Quick sort (array, low, high)

if low < high

Pivot = Partition (array)

Quick sort (array, low, Pivot - 1)

Quick sort (array, Pivot + 1, high)

Advantages:

- * simple code

- * easy to understand

Disadvantages:

- * can cause stack overflow when partitions become highly unbalanced.

Iterative Quick sort

Push (low, high)

while stack is not empty

low, high = POP()

Pivot = partition()

Push (left partition)

Push (right partition)

Advantages:

- * Better control over memory

- * lower risk of system stack overflow

Disadvantages:

- * More complicated to implement

Wojcik